
Running V FOODS Locally

A practical step-by-step setup guide for running the V FOODS frontend and backend on your local computer for development and testing.

1. Prerequisites

Before starting, make sure you have the following installed on your computer:

- **Node.js:** Version 18 or newer (comes with `npm`).
 - **Python:** Version 3.10 or newer (comes with `venv` and `pip`).
 - **Git:** For pulling the code and managing branches.
-

2. External Service Dependencies

V FOODS connects to three cloud services. To run the full system with live data, you will need credentials from each:

1. **Supabase Project:** Provides the PostgreSQL database, authentication, Row-Level Security, and Realtime WebSocket events.
 2. **PhonePe Merchant Sandbox:** Provides test payment gateway credentials (`MERCHANT_ID` , `SALT_KEY` , and `SALT_INDEX`) for testing wallet top-ups without spending real money.
 3. **MSG91 Account (Optional in Local Dev):** Provides SMS and WhatsApp messaging APIs for order ready notifications. If left blank, the app logs notifications to the console instead of sending live SMS.
-

3. Environment Variables Setup

Environment variables are configuration settings that tell the app where to connect without exposing private passwords in the code.

Frontend Configuration

Create a file named `.env` inside the `frontend/` directory with these variable names:

- `VITE_SUPABASE_URL` : The web address of your Supabase project (e.g. `https://xyzcompany.supabase.co`).
- `VITE_SUPABASE_ANON_KEY` : The public anonymous API key provided in your Supabase project settings.
- `VITE_API_URL` : The local web address of your running FastAPI backend (typically `http://localhost:8000`).

Backend Configuration

Create a file named `.env` inside the `backend/` directory with these variable names:

- `SUPABASE_URL` : The web address of your Supabase project.
 - `SUPABASE_SERVICE_ROLE_KEY` : The private service role key from Supabase (used by the backend for admin operations).
 - `PHONEPE_MERCHANT_ID` : Your PhonePe merchant ID code.
 - `PHONEPE_SALT_KEY` : The private cryptographic salt key provided by PhonePe for signing requests.
 - `PHONEPE_SALT_INDEX` : The salt index number assigned by PhonePe (usually `1`).
 - `PHONEPE_BASE_URL` : The API URL for PhonePe (sandbox: `https://api-preprod.phonepe.com/apis/pg-sandbox`).
 - `WEBHOOK_BASE_URL` : The address where PhonePe sends payment confirmations (e.g. a local tunnel address or public server URL).
 - `FRONTEND_URL` : The address of the frontend app (e.g. `http://localhost:5173`).
 - `MSG91_AUTH_KEY` : Your MSG91 authorization key for messaging.
 - `MSG91_WHATSAPP_NUM` : The verified WhatsApp sender number registered on MSG91.
 - `QR_SECRET` : A random secret passphrase used to generate secure QR codes for order collection.
-

4. Starting the Frontend (React + Vite)

1. Open your terminal and navigate to the `frontend` folder: `bash cd frontend`
 2. Install the necessary project dependencies: `bash npm install`
 3. Start the local development server: `bash npm run dev`
 4. The website will open in your browser at `http://localhost:5173`. You can log in using test credentials or create a new student account.
-

5. Starting the Backend (FastAPI Python)

1. Open a second terminal window and navigate to the `backend` folder: `bash cd backend`
 2. Create and activate a Python virtual environment:
 3. On Windows: `bash python -m venv venv .\venv\Scripts\activate`
 4. On macOS / Linux: `bash python3 -m venv venv source venv/bin/activate`
 5. Install required Python packages: `bash pip install -r requirements.txt`
 6. Start the FastAPI server: `bash uvicorn main:app --reload --port 8000`
 7. The API will start at `http://localhost:8000`. You can test endpoints directly in your browser using the built-in documentation at `http://localhost:8000/docs`.
-

6. Database Migrations Setup

1. Log in to your Supabase web dashboard.
2. Open the **SQL Editor**.
3. Run the SQL migration scripts located in the `supabase/migrations/` folder in sequential order (`001_...` , `002_...` , etc.).
4. This will create all required tables (`outlets` , `menu_items` , `orders` , `wallets` , etc.), configure Row-Level Security rules, and install the atomic wallet procedures.